

Detailed FS Functional Specification (FS)

DTTS API Processing & Reprocessing Application

1. Objective

Develop an RAP application with a beautiful and elegant user interface that enables users to:

- View DTTS transaction item data
- Edit non-success records

Introduce a edit button, once it is clicked only below fields from the displayed list should be in editable . This should be draft enabled.

header-frm_gln.
header-to_gln.
item-gtin
item-quantity
item-batch
item-shelf_expiry

- Reprocess selected records via API
- Create and process new records

Create a button called "Create" , on clicking that , the operation should be chosen from the popup then it should take to a new object page where the below mentioned fields should be asked to be filled.

header-frm_gln.
header-to_gln.
item-gtin
item-quantity
item-batch
item-shelf_expiry

This should have draft enabled .

this will take you new object page where all the fields to allow new records to the table with the Status NEW also the TRAN_ID for the table ZMM_SST_DTTSIT2 should be generated based on SRNO object

These new records will then be reprocessed as usual from the list page.

- Persist API responses back into the same table

Multi selection and reprocessing should be allowed.

at any point of time While writing the data in table zmm_sst_dttsit2

operation : abap.char(20);

frm_gln : abap.char(13);

to_gln : abap.char(13);

For new entries, take the value from the front end .

The **fully functional ABAP program is found in the location**

2. Data Source

Primary Table

ZMM_SST_DTTS_ITM

Each record represents:

One item → One PRODUCT entry in API payload

Header Dependency

Table: ZMM_SST_DTTS_HDR

Used to derive:

- FROMGLN (FRM_GLN)
 - AUTHGLN (TO_GLN)
-

3. Key Fields

The following fields are **keys and must never be editable**:

- MANDT
 - TRAN_ID
 - ITEM_NO
-

4. Business Fields

Relevant fields used in processing:

- GTIN
 - PROD_QTY
 - BATCH
 - EXP_DATE
 - PROD_STAT
 - TRANS_STAT
 - TR_RESPONSE
 - NOTIF_ID
-

5. Status Logic

```
SUCCESS → PROD_STAT = 'SUCCESS'  
ERROR    → PROD_STAT = 'ERROR'  
NEW      → PROD_STAT is initial or blank
```

6. Display Requirements

The application must display:

- TRAN_ID
- ITEM_NO
- GTIN
- BATCH
- EXP_DATE

- PROD_QTY
 - PROD_STAT
 - TRANS_STAT
 - NOTIF_ID
 - mat_doc
 - mat_year
-

7. Edit Rules

7.1 Editable Records

Only records where PROD_STAT $\neq$ 'SUCCESS' are editable

7.2 Field Editability

For non-success records:
All fields except key fields must be editable

7.3 Non-Editable Records

Records with PROD_STAT = 'SUCCESS' must be fully read-only

8. Record Creation

Users must be able to:

- Create new records manually

System must:

Set PROD_STAT = 'NEW' or initial

9. Reprocess / Process Logic

9.1 Trigger

- User selects one or more records
- Executes **Process / Reprocess** action

9.2 Mandatory Processing Rules

Rule 1: Skip Successful Records

Records with PROD_STAT = 'SUCCESS' must not be processed, even if selected

Rule 2: Grouping Logic

Selected records must be grouped by TRAN_ID

Rule 3: API Call Strategy

One TRAN_ID group = One API call

Rule 4: Payload Composition

Each API call must contain multiple PRODUCT entries for all non-success items of the same TRAN_ID

9.3 Processing Flow

1. User selects records
2. Remove records with PROD_STAT = 'SUCCESS'
3. Group remaining records by TRAN_ID
4. For each TRAN_ID group:
 - a. Read header (ZMM_SST_DTTS_HDR)
 - b. Collect all corresponding items
 - c. Build payload with multiple PRODUCT entries
 - d. Call API
 - e. Process response
 - f. Update database

9.4 No Processing Case

If all selected records are SUCCESS → No API call should be triggered

10. API Integration

10.1 Configuration Source

Table: `ZMM_DTTS_API_CON`

Used to retrieve:

- Proxy Class
- Method Name
- Request Structure
- Response Structure
- Logical Port

10.2 Payload Construction Rule

The payload must be constructed using the exact logic from the reference program.

10.3 Reference Logic

The system must **strictly follow the implementation pattern from the sample program** for:

- Payload construction
- Quantity formatting
- Date formatting
- GTIN conversion
- Product append logic

10.4 Mandatory Constraint

No alternative payload construction logic is allowed

11. API Execution

The API must be called using:

```
CALL METHOD lo_proxy->(method_name)
EXPORTING
    input = request
IMPORTING
    output = response.
```

12. Response Handling

12.1 Mandatory Rule

Response processing must follow the same logic as the reference program.

12.2 Processing Requirements

For each product in response:

- Update Notification ID
 - Update Response Code
 - Determine status (SUCCESS / ERROR)
 - Derive error description
 - Update transaction status
-
-
-

12.4 Audit Update

System must update:

- Changed Date
 - Changed Time
 - Changed By
-

13. Database Update

All re-processed records must be updated in below table only no other tables should be modified:

```
zmm_sst_dttsit2
```

14. Error Handling

System Errors

- Capture API exceptions
 - Record error message
-

Business Errors

- Use response code
 - Derive description
 - Update TRANS_STAT
-

15. Out of Scope

- Changes to API structure
 - Changes to proxy configuration
 - Changes to SAP standard posting logic
-

16. Final Instruction (CRITICAL FOR LLM)

The implementation must strictly reuse the API payload construction and response handling logic from the reference program. The application must only orchestrate selection, grouping, and execution, without altering the core API logic.

- Technology: SAP RAP (RESTful ABAP Programming Model)
 - UI: Fiori

MODULE : MM

i want you to guide me step by step and inch by inch , i will do copy paste only , you have to tell me how to create the if is a data definition or metadata or bahavior , everything you should guide step by step .

Also Add comment in details as much as possible , in behaviour , in CDS , in classes and in methods wherever a meaningful action is taking place

User the name space given below to create any object for RAP

Catalog – Z<MODULENAME><OBJNAME>CAT

GROUP – ZMODULENAME><OBJNAME>GRP

Keep the same name for the BAS Project and BSP application name

PROJ and BSP – Z<MODULENAME><OBJNAME>

ODATA Service - ZC<MODULENAME><OBJNAME>

CDS Views:

Base views – ZB<MODULENAME><OBJNAME>

Interface Views - ZI<MODULENAME><OBJNAME>

Root View - ZR<MODULENAME><OBJNAME>

Projection View - ZC<MODULENAME><OBJNAME>

Access Control View - ZA<MODULENAME><OBJNAME>

Service Def - ZSRV<MODULENAME><OBJNAME>

Service Binding - ZUI<MODULENAME><OBJNAME>

Database Table - Z<MODULENAME><OBJNAME>

Custom Fields – ZZ<FIELDNAME>

Custom Domain/DataElement – ZZ<OBJNAME>

Prog Name - Z<MODULENAME><OBJNAME>

Class Name - ZCL<MODULENAME><OBJNAME>

FM - Z<MODULENAME><OBJNAME>

The rest of the object should follow Z<MODULENAME><OBJNAME>

Create RAP program inside below folder . Create one if you do not find one.

The respective business objects should go inside respective folders.

Create the objects in .md files and follow the name space without fail

FS Folder

ZMM_DTTTS_COCKPIT (Folder)

BSP Library (Folder)

Business Services (Folder)

Core Data Services (Folder)

Dictionary (Folder)

Others (Folder)

Source Code Library (Folder)

Existing Data objects for your reference .

```
@EndUserText.label : 'DTTS Items - Current State Table'
@AbapCatalog.enhancement.category : #EXTENSIBLE_ANY
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #ALLOWED
define table zmm_sst_dttsit2 {

key mandt    : mandt not null;
key tran_id  : abap.char(20) not null;
@EndUserText.label : 'Item Number'
key item_no  : abap.numc(4) not null;
zeile       : mblpo;
product     : matnr;
prod_name   : maktx;
@Semantics.quantity.unitOfMeasure : 'zmm_sst_dttsit2.prod_unit'
prod_qty    : menge_d;
prod_unit   : meins;
gtin        : z_dgtin;
batch       : zmm_sst_batch;
exp_date    : zmm_sst_expiry_date;
notif_id    : zmm_sst_notif_id;
tr_response : zmm_sst_response;
mat_doc     : mblnr;
mvt_type    : bwart;
sr_number   : zmm_br_serial_no;
created_date : dats;
created_time : tims;
created_by  : xubname;
changed_date : dats;
changed_time : tims;
changed_by  : xubname;
prod_stat   : zmm_sst_prod_stat;
@EndUserText.label : 'Transaction Status'
trans_stat  : abap.char(1333);
@EndUserText.label : 'Operation'
operation   : abap.char(20);
```

```

frm_gln    : abap.char(13);
to_gln     : abap.char(13);
}

```

```

@EndUserText.label : 'DTTS integration - serial to batch - Header Table'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #ALLOWED
define table zmm_sst_dtts_hdr {

key mandt    : mandt not null;
key tran_id  : ztran_id not null;
mat_doc     : mblnr;
doc_yr      : mjahr;
gjhr        : gjahr;
mvt_type    : bwart;
budat       : budat;
@EndUserText.label : 'Operation'
operation    : abap.char(20);
frm_gln     : zmm_sst_gln_src;
to_gln      : zmm_sst_gln_des;
auth_gln    : zmm_sst_gln_auth;
created_date : dats;
created_time : tims;
created_by   : xubname;
changed_date : dats;
changed_time : tims;
changed_by   : xubname;
@EndUserText.label : 'Status'
status      : abap.char(10);
}

```

```

@EndUserText.label : 'DTTS integration - serial to batch - Item Table'
@AbapCatalog.enhancement.category : #EXTENSIBLE_ANY
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A

```

```

@AbapCatalog.dataMaintenance : #ALLOWED
define table zmm_sst_dtts_itm {

key mandt    : mandt not null;
key tran_id  : ztran_id not null;
@EndUserText.label : 'Item Number'
key item_no  : abap.numc(4) not null;
zeile       : mblpo;
product      : matnr;
prod_name    : maktx;
@Semantics.quantity.unitOfMeasure : 'zmm_sst_dtts_itm.prod_unit'
prod_qty     : menge_d;
prod_unit    : meins;
gtin         : z_dgtin;
batch        : zmm_sst_batch;
exp_date     : zmm_sst_expiry_date;
notif_id     : zmm_sst_notif_id;
tr_response  : zmm_sst_response;
mat_doc      : mblnr;
mvt_type     : bwart;
sr_number    : zmm_br_serial_no;
created_date : dats;
created_time : tims;
created_by   : xubname;
changed_date : dats;
changed_time : tims;
changed_by   : xubname;
prod_stat    : zmm_sst_prod_stat;
@EndUserText.label : 'Transaction Status'
trans_stat   : abap.char(1333);
}

```

```

@AbapCatalog.sqlViewName: 'ZVIMMDTTSOP'
@AbapCatalog.compiler.compareFilter: true
@AbapCatalog.preserveKey: true
@ClientHandling.algorithm: #SESSION_VARIABLE
@EndUserText.label: 'DTTS Items with Header Operation'

```

```

define view ZIMMDTTS_4
as select from ZIMM_DTTS_FY_HELPER as itm
left outer join zmm_sst_dtts_hdr as hdr
on itm.mandt = hdr.mandt
and itm.tran_id = hdr.tran_id
{
key itm.mandt,
key itm.tran_id,
key itm.item_no,

```

```

    itm.zeile,
    itm.product,
    itm.prod_name,
    itm.prod_qty,
    itm.prod_unit,
    itm.gtin,
    itm.batch,
    itm.exp_date,
    itm.notif_id,
    itm.tr_response,
    itm.mat_doc,
    itm.mvt_type,
    itm.sr_number,
    itm.created_date,
    itm.created_time,
    itm.created_by,
    itm.changed_date,
    itm.changed_time,
    itm.changed_by,
    itm.prod_stat,
    itm.trans_stat,
    itm.fiscal_year,
    itm.DOC_YEAR,
    itm.operation as item_operation,
    hdr.operation as header_operation

```

```

}
```